

Normalization is a database design technique used to organize data into multiple related tables so that data redundancy (unnecessary repetition) is reduced and data anomalies are prevented.

Simple language:

Normalization = Data ko logically chhoti-chhoti related tables me divide karna, taaki same data baar-baar store na ho aur Insert, Update, Delete ke time problem na aaye.

Example:

Suppose ek college database me hum ye table banate hain:

Student_ID	Student_Name	Course_ID	Course_Name	Teacher
101	Tarun	C01	DBMS	Sharma
102	Rahul	C01	DBMS	Sharma
103	Aman	C01	DBMS	Sharma
104	Ravi	C02	Java	Verma

Yaha **DBMS** aur **Sharma** baar-baar repeat ho rahe hain.

Ye **data redundancy** hai.

Normalization ka goal hai:

Student
↓
Enrollment
↓
Course

Instead of putting everything in one huge table.

2. Normalization ki zarurat kyun padi?

Agar hum saara data ek hi table me rakhte hain, to mainly ye problems aa sakti hain:

1. Data Redundancy
2. Insertion Anomaly
3. Update Anomaly
4. Deletion Anomaly

Aur redundancy ko samajhne ke liye **row-level duplication** aur **column-level duplication/repetition** ko alag samajhna useful hai.

3. Data Redundancy kya hai?

Redundancy = same information ka unnecessarily multiple times store hona.

Example:

Student_ID	Name	Course	Teacher
101	Tarun	DBMS	Sharma
102	Rahul	DBMS	Sharma
103	Aman	DBMS	Sharma

Yaha:

DBMS
Sharma



multiple rows me repeat ho rahe hain.

Agar 500 students DBMS padhte hain:



Agar 500 students DBMS padhte hain:

DBMS
Sharma
DBMS
Sharma
DBMS
Sharma
...

500 times information store hogi.

Isse:

- storage waste hota hai
- inconsistency ka chance badhta hai
- update difficult hota hai

4. Row-level duplication kya hai?

Row-level duplication ka matlab hai ki **poori row ya almost same row database me multiple times store ho jaye.**

Example:

ID	Name	Course
101	Tarun	DBMS
102	Rahul	Java
101	Tarun	DBMS

Last row first row ki duplicate hai.

Yaha:

101 | Tarun | DBMS



same row dobara aa gayi.

Problem

Agar duplicate rows aa gayi to:

- database size unnecessarily badhega
- counting galat ho sakti hai
- update/delete karte waqt confusion ho sakta hai

Important

Primary Key normally exact row duplication ko prevent karne me help karti hai.

For example:

SQL

```
CREATE TABLE Student (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

5. Column-level duplication / repetition

Ye thoda different concept hai.

Yaha **same information different rows me repeatedly stored hoti hai**, because table me multiple entities ki information unnecessarily mix kar di gayi hai.

Example:

Student_ID	Student_Name	Course_ID	Course_Name	Teacher
101	Tarun	C01	DBMS	Sharma
102	Rahul	C01	DBMS	Sharma
103	Aman	C01	DBMS	Sharma

Course_Name column me:

DBMS
DBMS
DBMS



Database part 1

`Course_Name` column me:

DBMS
DBMS
DBMS



aur `Teacher` me:

Sharma
Sharma
Sharma



repeat ho raha hai.

Ye **redundant data** hai.

Important distinction

Technically database theory me "**column-level duplication**" koi standard formal anomaly term nahi hai. Interview/exam ke context me ise generally **repeated/redundant values in a column** ke sense me samjha ja sakta hai.



7. Insertion Anomaly

Definition

Insertion anomaly occurs when we cannot insert new information into the database without having to provide some unrelated or unnecessary information.

Example:

Student_ID	Student_Name	Course_ID	Course_Name	Teacher
101	Tarun	C01	DBMS	Sharma
102	Rahul	C02	Java	Verma

Suppose college ne ek **new course** introduce kiya:

C03 = Python
Teacher = Gupta



Lekin abhi **koi student Python me enrolled nahi hai.**

Hum kya insert karen?

```
Student_ID = ?
Student_Name = ?
C03
Python
Gupta
```

Student_ID aur **Student_Name** required hain, lekin student abhi hai hi nahi.

To hum course ki information independently store nahi kar pa rahe.

This is insertion anomaly.

7/1/2023 3:00:00 PM

Normalization ke baad

Hum tables separate kar dete hain:

Student

Student_ID	Name
------------	------

101	Tarun
-----	-------

102	Rahul
-----	-------

Course

Course_ID	Course_Name	Teacher
-----------	-------------	---------

C01	DBMS	Sharma
-----	------	--------

C02	Java	Verma
-----	------	-------

C03	Python	Gupta
-----	--------	-------

Ab Python course easily insert ho gaya.



8. Update Anomaly

Definition

Update anomaly occurs when the same piece of information is stored in multiple rows, so changing it requires updating multiple records.

Example:

Student_ID	Student	Course	Teacher
101	Tarun	DBMS	Sharma
102	Rahul	DBMS	Sharma
103	Aman	DBMS	Sharma

Suppose DBMS teacher change ho gaya:

Sharma → Gupta



Ab hume 3 rows update karni padengi:

Ab hume 3 rows update karni padengi:

Row 1 → Gupta
Row 2 → Gupta
Row 3 → Gupta

Agar galti se ek row update nahi hui:

Student_ID	Student	Course	Teacher
101	Tarun	DBMS	Gupta
102	Rahul	DBMS	Sharma ✖
103	Aman	DBMS	Gupta

Ab same DBMS ke **do teachers** dikh rahe hain.

Database inconsistent ho gaya.

Ye **update anomaly** hai.



Normalization

Course ko separate table me rakho:

Course_ID	Course	Teacher
C01	DBMS	Gupta

Ab teacher ko sirf **one place** par update karna padega.

9. Deletion Anomaly

Definition

Deletion anomaly occurs when deleting one piece of information unintentionally causes another useful piece of information to be lost.

Example:

Student_ID	Student	Course_ID	Course
101	Tarun	C01	DBMS
102	Rahul	C02	Java

Suppose Rahul college chhod deta hai.

Hum Rahul ki row delete karte hain:

`</> SQL`

```
DELETE FROM StudentCourse
WHERE Student_ID = 102;
```

Row delete ho gayi:

Student_ID	Student	Course_ID	Course
101	Tarun	C01	DBMS

Ab `C02 = Java` ki information bhi gayab ho gayi.

Lekin Java course abhi bhi college me exist karta ho sakta hai.

Student ko delete karne ke saath course information bhi delete ho gayi.

This is **deletion anomaly**.

Separate tables:

Student

Student_ID	Name
101	Tarun

Course

Course_ID	Course
C01	DBMS
C02	Java

Enrollment

Student_ID	Course_ID
101	C01



Ab Rahul ki enrollment delete karne par Java course delete nahi hoga.

10. Ek complete example

Bad design:

Student_ID	Student	Course_ID	Course	Teacher
101	Tarun	C01	DBMS	Sharma
102	Rahul	C01	DBMS	Sharma
103	Aman	C02	Java	Verma

Problems:

Redundancy

DBMS → repeated
Sharma → repeated



Insertion anomaly

New course without student insert karna difficult.

Update anomaly

Sharma ka naam change karne ke liye multiple rows update karni hongi.

Deletion anomaly

Last student ko delete karne par course ki information bhi disappear ho sakti hai.

-20205e5096uu

11. Normalization ke baad

Hum tables divide kar sakte hain:

Student Table

Student_ID	Student_Name
101	Tarun
102	Rahul
103	Aman

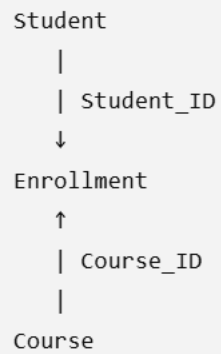
Course Table

Course_ID	Course_Name	Teacher
C01	DBMS	Sharma
C02	Java	Verma

Enrollment Table

Student_ID	Course_ID
101	C01
102	C01
103	C02

Ab:



Enrollment Table

Student_ID

Data duplication kaafi reduce ho gayi.



12. Normalization ka main goal

Yaad rakhne ke liye:

Normalization ka main purpose data redundancy ko reduce karna aur insertion, update aur deletion anomalies ko prevent karna hai.

Short formula:

```
Normalization
  ↓
Reduce Redundancy
  ↓
Better Table Design
  ↓
Prevent Anomalies
  ↓
Data Consistency
```

1NF (First Normal Form) kya hai?

1NF = First Normal Form

1NF normalization ka **first step** hai. Iska main purpose table ke andar **repeating/multiple values** ko remove karke **atomic (single) values** rakhna hai.

Simple definition

A table is in 1NF if every column contains atomic (single) values, there are no repeating groups, and each row can be uniquely identified.

Simple language me:

Har cell me sirf ek value honi chahiye.

✗ 1NF me nahi hai

Suppose:

Student_ID	Name	Phone
101	Tarun	9876, 8765
102	Rahul	7654

Problem:

Phone ke ek cell me **2 values** hain:

9876, 8765

Ye atomic value nahi hai.

✓ 1NF me hai

Phone numbers ko separate rows me rakho:

Student_ID	Name	Phone
101	Tarun	9876
101	Tarun	8765
102	Rahul	7654

Ab har cell me **single value** hai.

Ek aur example

✗ Not 1NF

Student_ID	Name	Courses
101	Tarun	C++, Java, Python
102	Rahul	DBMS, SQL

Courses column me multiple values hain.

C++, Java, Python

Ek cell me 3 values → 1NF violate

Database part 1

Student_ID	Name	Course
101	Tarun	C++
101	Tarun	Java
101	Tarun	Python
102	Rahul	DBMS
102	Rahul	SQL

Ab:

Course

↓

C++

Java

Python

DBMS

SQL

↓

Har cell me **one value**.

Rule 2: Repeating groups nahi hone chahiye

✗ Aisa design:

ID	Name	Phone1	Phone2	Phone3
101	Tarun	9876	8765	7654

Phone1 , Phone2 , Phone3 repeating group ko show karte hain.

Better design:

ID	Name	Phone
101	Tarun	9876
101	Tarun	8765
101	Tarun	7654

Rule 3: Rows uniquely identifiable honi chahiye

Usually table me **Primary Key** use ki jaati hai taaki har row uniquely identify ho sake.

Example:

Student_ID	Name
101	Tarun
102	Rahul

Student_ID unique hai.

2NF (Second Normal Form) kya hai?

2NF = Second Normal Form

2NF ko samajhne ke liye **1NF + Partial Dependency** samajhna sabse important hai.

Simple definition

A table is in 2NF if it is already in 1NF and every non-key attribute is fully dependent on the entire primary key, not just a part of the primary key.

Simple language:

2NF = 1NF + No Partial Dependency

1. Pehle Partial Dependency samjho

Partial dependency tab hoti hai jab **Primary Key composite ho** (2 ya more columns) aur koi non-key column **poori key ke bajay key ke sirf ek part par depend kare**.

Example:

Student_ID	Course_ID	Student_Name	Course_Name	Marks	
101	C01	Tarun	DBMS	80	
101	C02	Tarun	Java	75	
102	C01	Rahul	DBMS	90	

Suppose Primary Key hai:

(Student_ID, Course_ID)



Yani dono milkar row ko uniquely identify karte hain.

Student_ID + Course_ID



Marks

Marks ko dono ki zarurat hai:

Student_ID + Course_ID → Marks

Lekin:

Student_ID → Student_Name

Student_Name ko Course_ID ki zarurat nahi hai.

Similarly:

Course_ID → Course_Name

Course_Name ko Student_ID ki zarurat nahi hai.



Ye partial dependency hai.

2. Problem kya hai?

Same student multiple courses karega to uska name baar-baar repeat hoga:

Student_ID	Course_ID	Student_Name
101	C01	Tarun
101	C02	Tarun
101	C03	Tarun

Tarun repeatedly store ho raha hai.

Similarly:

Course_ID	Course_Name
C01	DBMS
C01	DBMS
C02	Java

DBMS bhi repeat ho raha hai.

Ye redundancy aur anomalies create kar sakti hai.

Hum table ko divide karenge.

Student Table

Student_ID	Student_Name
101	Tarun
102	Rahul

Dependency:

Student_ID → Student_Name

Course Table

Course_ID	Course_Name
C01	DBMS
C02	Java

Dependency:

Course_ID → Course_Name

Enrollment Table

Student_ID	Course_ID	Marks
101	C01	80
101	C02	75
102	C01	90

Yaha:

(Student_ID, Course_ID) → Marks

Marks **poori composite key** par dependent hai.

Ab partial dependency nahi hai.

Therefore → **2NF**.

4. 1NF vs 2NF

1NF	2NF
Atomic values	1NF hona compulsory
One cell → One value	No partial dependency
Repeating groups remove	Composite key ke partial dependency remove
Basic normalization	Next level normalization

Yaad rakhna:

1NF
↓
Atomic Values
↓
2NF
↓
No Partial Dependency



3NF (Third Normal Form)

A table is in 3NF if:

1. It is already in 2NF.
2. There is no transitive dependency.
3. Every non-key attribute depends directly on the primary key, not on another non-key attribute.

Simple formula:

3NF = 2NF + No Transitive Dependency

Example

Suppose:

Student_ID	Student_Name	Dept_ID	Dept_Name
101	Tarun	D01	CSE
102	Rahul	D01	CSE

1. Transitive Dependency kya hai?

Suppose table:

Student_ID	Student_Name	Dept_ID	Dept_Name
101	Tarun	D01	CSE
102	Rahul	D01	CSE
103	Aman	D02	IT

Primary Key:

Student_ID

Dependencies:

Student_ID → Student_Name

Student_ID → Dept_ID

Dept_ID → Dept_Name

↓

Notice:

```
Student_ID
  ↓
Dept_ID
  ↓
Dept_Name
```

`Dept_Name` directly `Student_ID` par depend nahi kar raha.

Wo `Dept_ID` par depend kar raha hai.

Yani:

```
Student_ID → Dept_ID → Dept_Name
```

Ye **Transitive Dependency** hai.

2. Problem kya hai?

Dekho `CSE` baar-baar repeat ho raha hai:

Student_ID	Student	Dept_ID	Dept_Name
101	Tarun	D01	CSE
102	Rahul	D01	CSE
103	Aman	D02	IT
104	Ravi	D01	CSE

Agar `D01` department ka naam change hota hai:

```
CSE → Computer Science
```

to hume multiple rows update karni padengi.

Agar ek row update nahi hui:

Student_ID	Dept_ID	Dept_Name
101	D01	Computer Science
102	D01	CSE ✖
104	D01	CSE ✖

Database inconsistent ho jayega.

3. 3NF kaise achieve karenge?

Original table:

Student_ID	Student_Name	Dept_ID	Dept_Name
101	Tarun	D01	CSE
102	Rahul	D01	CSE
103	Aman	D02	IT

Isko 2 tables me divide kar do.

Student Table

Student_ID	Student_Name	Dept_ID
101	Tarun	D01
102	Rahul	D01
103	Aman	D02



Department Table

Dept_ID	Dept_Name
---------	-----------

D01	CSE
-----	-----

D02	IT
-----	----

Ab dependency:

Student_ID → Student_Name

Student_ID → Dept_ID

Dept_ID → Dept_Name

Dept_Name ab Student table me directly nahi hai.

Isliye **transitive dependency remove** ho gayi.

4. 2NF vs 3NF

Ye interview me bahut important hai:

2NF	3NF
1NF hona chahiye	2NF hona chahiye
Partial dependency remove	Transitive dependency remove
Composite key ke part par dependency nahi	Non-key attribute → non-key attribute dependency nahi
Example: Student_ID → Student_Name when key = (Student_ID, Course_ID)	Example: Dept_ID → Dept_Name

4. 2NF vs 3NF

Ye interview me bahut important hai:

2NF	3NF
1NF hona chahiye	2NF hona chahiye
Partial dependency remove	Transitive dependency remove
Composite key ke part par dependency nahi	Non-key attribute $\rightarrow$ non-key attribute dependency nahi
Example: <code>Student_ID $\rightarrow$ Student_Name</code> when key = <code>(Student_ID, Course_ID)</code>	Example: <code>Dept_ID $\rightarrow$ Dept_Name</code>

BCNF kya hai?

BCNF = Boyce-Codd Normal Form

BCNF ko 3NF ka stronger version samjho.

Simple definition

A relation is in BCNF if for every functional dependency `$X \rightarrow Y$` , `X` must be a superkey.

Simple language:

Jis attribute/set of attributes se kisi doosre attribute ki value determine ho rahi hai, woh attribute/set superkey hona chahiye.

Yaad rakho:

1NF → Atomic values
2NF → No Partial Dependency
3NF → No Transitive Dependency
BCNF → Every determinant must be a Superkey

Determinant = Functional dependency ke left side ka attribute/set.

Example:

$A \rightarrow B$

Yaha **A = determinant**.

BCNF kehta hai:

$A \rightarrow B$
↓
A must be a Superkey



2. Superkey kya hoti hai?

Superkey wo attribute ya attributes ka set hai jo table ki row ko uniquely identify kar sake.

Example:

Student_ID	Name	Email
101	Tarun	<u>tarun@gmail.com</u> ↗
102	Rahul	<u>rahul@gmail.com</u> ↗

Possible superkeys:

```
Student_ID
Email
Student_ID + Name
Student_ID + Email
```



`Student_ID` aur `Email` individually unique hain.

1. Superkey kya hoti hai?

Superkey = ek ya multiple attributes ka aisa set jo table ki har row ko uniquely identify kare.

Example

Student_ID	Name	Email	Phone
101	Tarun	tarun@gmail.com ↗	9876
102	Rahul	rahul@gmail.com ↗	8765
103	Aman	aman@gmail.com ↗	7654

Maan lo:

- Student_ID unique hai
- Email unique hai
- Phone unique hai

To ye **superkeys** hain:



Database part 1

Student_ID ✓
Email ✓
Phone ✓



Student_ID + Name ✓
Student_ID + Email ✓
Email + Phone ✓
Student_ID + Name + Email ✓

Lekin important point

Student_ID + Name superkey hai, **lekin minimal nahi hai**, kyunki sirf Student_ID hi row identify kar sakta hai.

Isliye:

Superkey:
Student_ID
Student_ID + Name
Student_ID + Email
Email + Phone



Candidate Key kya hogi?

Candidate Key = minimal superkey.

Is example me:

Student_ID → Candidate Key
Email → Candidate Key
Phone → Candidate Key

Relationship:

Candidate Key $\subset$ Superkey

Har candidate key superkey hoti hai, lekin har superkey candidate key nahi hoti.

2. Ab BCNF ka example

BCNF samajhne ke liye ye example bahut achha hai:

Student-Course-Instructor

Student	Course	Instructor
Tarun	DBMS	Sharma
Rahul	DBMS	Sharma
Aman	Java	Verma
Ravi	Java	Verma

Assume college ke rules:

Rule 1

Ek student ek course me ek instructor ke under hai.

$(\text{Student}, \text{Course}) \rightarrow \text{Instructor}$

Rule 2

Ek instructor sirf **ek hi course** padhata hai.

$\text{Instructor} \rightarrow \text{Course}$

3. Candidate Key kya hai?

Pehle $(\text{Student}, \text{Course})$ dekho.

$\text{Student} + \text{Course} \rightarrow \text{Instructor}$



Database part 1

Agar:

Student = Tarun
Course = DBMS

to instructor:

Sharma

mil jayega.

So:

(Student, Course)

row ko uniquely identify kar sakta hai.

Therefore:

(Student, Course) = Candidate Key

4. Ab BCNF ka main point

Functional dependency:

Instructor → Course



Yani:

Sharma → DBMS

Verma → Java



Ab check karo:

Kya **Instructor** superkey hai?

No ❌

Kyun?

Sharma se hume course **DBMS** pata chal jayega, lekin student kaun hai ye nahi pata.

Sharma ke saath:

Tarun
Rahul

dono students ho sakte hain.

Therefore **Instructor** row ko uniquely identify nahi kar raha.

So:

Instructor → Course
↓
Instructor is NOT Superkey ❌

BCNF ke according:

Every determinant must be a superkey.

Yaha determinant = **Instructor** .

Instructor superkey nahi hai.



Therefore:

✗ Table BCNF me nahi hai.

5. BCNF me kaise convert karenge?

Original table:

Student	Course	Instructor
Tarun	DBMS	Sharma
Rahul	DBMS	Sharma
Aman	Java	Verma
Ravi	Java	Verma

Isko 2 tables me divide karenge.

Instructor_Course

Instructor	Course
------------	--------

Sharma	DBMS
--------	------

Verma	Java
-------	------

Yaha:

Instructor → Course

Aur **Instructor** ab is table ki **key** hai.

Therefore:

Instructor = Superkey 

Student_Instructor

Student	Instructor
---------	------------

Tarun	Sharma
-------	--------

Rahul	Sharma
-------	--------

Aman	Verma
------	-------

Ravi	Verma
------	-------

Ab unnecessary repetition reduce ho gayi aur BCNF condition satisfy ho sakti hai.

Denormalization kya hai?

Denormalization = Normalized database me intentionally kuch redundant data ko wapas store karna, taaki data retrieval/query performance improve ho.

Simple language:

Normalization me hum data ko tables me todte hain aur redundancy kam karte hain.
Denormalization me performance ke liye kuch tables/data ko combine karke redundancy intentionally allow karte hain.

1. Normalization vs Denormalization

Normalized database

Suppose:

Student

Student_ID	Name	Dept_ID
101	Tarun	D01
102	Rahul	D01

Department

Dept_ID	Dept_Name
D01	CSE

Agar hume student ke saath department name chahiye:

Agar hume student ke saath department name chahiye:

</> SQL



```
SELECT Student.Name, Department.Dept_Name
FROM Student
JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

Yaha JOIN karna padega.

2. Denormalization

Performance improve karne ke liye hum department name ko Student table me bhi store kar sakte hain:

Student_ID	Name	Dept_ID	Dept_Name	
101	Tarun	D01	CSE	
102	Rahul	D01	CSE	

Ab:

```
</> SQL  
  
SELECT Name, Dept_Name  
FROM Student;
```

JOIN ki zarurat nahi.

Lekin **CSE** multiple rows me repeat ho raha hai.

```
CSE  
CSE  
CSE  
CSE  
...
```

Ye intentional redundancy hai.

3. Denormalization kyun karte hain?

Main reason:

Query performance improve karna

Agar application ko baar-baar multiple tables ko JOIN karke data nikalna pad raha hai, to kuch commonly required data ko same table/document me store kar sakte hain.

Benefits:

- JOINS kam ho sakte hain
- Read queries faster ho sakti hain
- Reporting/analytics queries simpler ho sakti hain
- Frequently accessed data ek jagah mil sakta hai

4. Lekin problem bhi hai

Denormalization ka disadvantage:

Update Anomaly

Agar department ka naam:

CSE → Computer Science

change hua, to multiple student records update karne padenge.

Agar:

Student	Dept
Tarun	Computer Science
Rahul	CSE ✖
Aman	Computer Science



to inconsistency aa jayegi.

5. Normalization vs Denormalization

Normalization	Denormalization
Redundancy reduce karta hai	Redundancy intentionally increase kar sakta hai
Data consistency better	Consistency maintain karna harder
Tables generally separate	Tables/data combine ho sakta hai
JOINS zyada ho sakte hain	JOINS kam ho sakte hain
Write/update operations efficient ho sakte hain	Read operations faster ho sakte hain
OLTP systems me common	Reporting/analytics/read-heavy systems me useful